

Templates

Generic Programming Mastery



Type-Independent Blueprints for Modern C++

WRITE ONCE • SCALE ANYWHERE • COMPILE-TIME MAGIC

1. The Blueprint Concept

In standard programming, you often have to write the same logic multiple times for different data types (int, float, double). **Templates** solve this by allowing you to define a "generic" version of a function or class.

Standard Analogy: The Mold

Imagine a factory mold for a cup. Whether you pour in liquid plastic, molten glass, or ceramic clay, the **shape** remains the same. The mold (Template) is independent of the material (Data Type).

Why Templates?

- **DRY Principle:** Don't Repeat Yourself. One code block handles all types.
- **Performance:** Templates are resolved at compile-time, so they are as fast as manually written code.
- **Type Safety:** Unlike `void*`, templates maintain the integrity of the data type.

2. Function Templates

A function template allows a single function to process different data types using placeholder names (usually T).

```
template <typename T>
T findMax(T a, T b) {
    return (a > b) ? a : b;
}
```

The Logic:

The compiler looks at the arguments you pass. If you pass `int`, it generates an integer version of the function. If you pass `double`, it generates a double version. This is called **Instantiation**.

3. Generic Classes

Class templates are used when a whole class can benefit from being type-independent. This is how the C++ Standard Template Library (STL) works (e.g., `vector<int>` vs `vector<string>`).

```
template <class T>
class Container {
private:
    T item;
public:
    void add(T i) { item = i; }
    T get() { return item; }
};
```

Usage: You specify the type inside angle brackets `<>` when creating the object.

4. Practical Integration

Applying generic logic to the CodeHive ecosystem allows us to handle different scoring systems (Percentages as `int` vs GPA as `double`) using one class.

```
template <typename T>
class CodeHive {
public:
    T score;
    void print() { cout << "Grade: " << score; }
};

int main() {
    CodeHive<int> basic;
    basic.score = 90;

    CodeHive<string> letter;
    letter.score = "A+";
}
```

5. Multiple Template Parameters

Templates are not limited to just one placeholder. You can use multiple types if your data involves complex pairs.

```
template <typename T, typename U>
class Entry {
public:
    T id;
    U name;

    Entry(T i, U n) : id(i), name(n) {}
};

int main() {
    // ID as int, Name as string
    Entry<int, string> user1(101, "Avni");
}
```

6. Under the Hood: Resolving T

It is a common misconception that templates make the program slow. In reality, templates are **Static**.

When the compiler sees `add<int>(5, 10)`, it literally writes a new function in machine code specifically for integers. This is why templates are found in high-performance engines.

Template Specialized Rule:

If you need a template to behave differently specifically for one type (like `char*`), C++ allows **Template Specialization**.

7. Why It Beats Traditional C

1. No Type Casting: You don't need to cast `void*` back and forth, which is a major source of bugs in C.

2. Clean Syntax: Code is more readable because it focuses on logic rather than data management.

3. Built-in Optimizer: Since the compiler knows the type exactly during generation, it can inline the code more efficiently than virtual functions.

8. Comparison Table

Feature	Overloading	Templates
Definition	Multiple separate functions	Single generic function
Code Size	Manual repetition	Auto-generated by compiler
Flexibility	Limited to specific types	Works with any type

Pro-Tip: Use templates for containers and mathematical operations. Use overloading for logic that changes significantly between types.

9. Summary & Skill Lab

- **Placeholder:** T is the most common name for types.
- **Instantiation:** The process of generating code for a type.
- **typename vs class:** In template declarations, they are interchangeable.
- **STL:** Rebuilds the world using templates.

Challenge Task:

The Dynamic Box: Create a class template `Box<T>` that has a function `swapItems(T &a, T &b)`. Test it by swapping two integers and then two strings.

GENERIC LOGIC DEPLOYED: TEMPLATES MASTERED